```
================================================================================
🧪 ЗАПУСК ГЛОБАЛЬНОГО ТЕСТА: SOFTMAX (FORWARD + BACKWARD)
================================================================================
Тестирование на устройстве: NVIDIA GeForce RTX 2060

Вычисление Softmax Forward на GPU...
32
Traceback (most recent call last):
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_nncl_glob.py", line 155, in <module>
    test_softmax_glob_pipeline()
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_nncl_glob.py", line 120, in test_softmax_glob_pipeline
    y_cl = nnclglob.softmax(x_cl)
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_glob.py", line 90, in softmax
    self.prg_softmax.softmax_forward(
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
        self.queue,
        ^^^^^^^^^^^
    ...<4 lines>...
        np.int32(len_)
        ^^^^^^^^^^^^^^
    )
    ^
  File "<generated: 'pyopencl invoker for 'softmax_forward'-2886f10916a12aa6'>", line 7, in enqueue_knl_softmax_forward
pyopencl._cl.LogicError: clSetKernelArg failed: INVALID_MEM_OBJECT - when processing arg#1 (1-based):
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>
```

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > cl_kernels > Softmax.cl

```
__kernel void softmax_forward(
    __global const float* X, // Входная матрица (например, скоры внимания) [B * H * L, SEQ_LEN]
    __global float* Y,       // Выходной тензор вероятностей [B * H * L, SEQ_LEN]
    const int seq_len)
{
    int row = get_group_id(0);
    int tid = get_local_id(0);
    int gsize = get_local_size(0);

    __local float l_mem[256]; // Общая локальная память под редукции максимума и суммы

    // 1. Поиск максимума строки (Safe Softmax)
    float local_max = -INFINITY;
    for (int i = tid; i < seq_len; i += gsize) {
        float val = X[row * seq_len + i];
        if (val > local_max) local_max = val;
    }
    l_mem[tid] = local_max;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (int s = gsize / 2; s > 0; s >>= 1) {
        if (tid < s) {
            if (l_mem[tid + s] > l_mem[tid]) l_mem[tid] = l_mem[tid + s];
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    float max_val = l_mem[0];
    barrier(CLK_LOCAL_MEM_FENCE); // Переиспользуем локальную память

    // 2. Вычисление экспонент и их суммы
    float local_sum = 0.0f;
    for (int i = tid; i < seq_len; i += gsize) {
        int idx = row * seq_len + i;
        float e = exp(X[idx] - max_val);
        Y[idx] = e; // Временно пишем экспоненту в выходной массив
        local_sum += e;
    }
    l_mem[tid] = local_sum;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (int s = gsize / 2; s > 0; s >>= 1) {
        if (tid < s) l_mem[tid] += l_mem[tid + s];
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    float sum_val = l_mem[0];

    // 3. Финальное деление на сумму
    for (int i = tid; i < seq_len; i += gsize) {
        int idx = row * seq_len + i;
        Y[idx] /= sum_val;
    }
}
__kernel void softmax_backward(
```

```
    def softmax(self, x: Tensor, dim: int = -1) -> Tensor:
        """
        Глобальная операция Safe Softmax (Forward)
        Рассчитана на работу со скорами внимания [B * H * L, SEQ_LEN]
        """
        assert dim == -1 or dim == len(x.shape) - 1, "Текущая реализация ядра Softmax поддерживает редукцию только по последней оси."

        seq_len = x.shape[-1]
        len_=seq_len
        print(seq_len)
        num_rows = x.size // seq_len

        # Создаем выходной тензор (память во VRAM выделится автоматически)
        y = Tensor(x.shape, self.ctx, self.queue)

        # Настройка сетки потоков
        local_size = (256,)
        global_size = (num_rows * 256,)

        # Запускаем ядро, явно проверяя, что передаем нативные cl.Buffer объекты
        self.prg_softmax.softmax_forward(
            self.queue,
            global_size,
            local_size,
            x.data_buffer,
            y.data_buffer,
            np.int32(len_)
        )
        self.queue.finish()
        return y
```

ошибка ругается на последний аргумент,что странно.